Java Day 8：面向对象进阶——封装

学习目标：掌握 private、public、getter、setter，并用参数校验保护性能测试数据

一、今天的学习安排

环节	内容	建议时间
1	理解封装的作用	20 分钟
2	private 与 public	20 分钟
3	getter 与 setter	25 分钟
4	setter 参数校验	20 分钟
5	性能测试实战与作业	35 分钟

二、为什么需要封装?

之前的 HttpResult 类中，属性可以被外部代码直接修改。如果 responseTime 被赋值为 -100，程序仍可能接受它，后续平均响应时间等统计就可能失真。封装的目的，是保护对象内部数据，并规定外部访问和修改数据的入口。

```
public class HttpResult {
    int responseTime;
    int statusCode;
}

HttpResult result = new HttpResult();
result.responseTime = -100; // Invalid value is not blocked
```

三、private 与 public

public：公共访问，其他类可以访问。private：私有访问，只能在当前类内部直接访问。将属性声明为 private 后，外部不能再通过 result.responseTime 直接读写。

```
public class HttpResult {
    private int responseTime;
    private int statusCode;
}

// Outside the class:
// result.responseTime = 300; // Compile error: private field
```

四、getter 与 setter

getter 用于读取属性，通常以 get + 属性名命名；setter 用于修改属性，通常以 set + 属性名命名。

```
// getter: read the response time
public int getResponseTime() {
    return responseTime;
}

// setter: update the response time
public void setResponseTime(int responseTime) {
    this.responseTime = responseTime;
}

// Usage
int time = result.getResponseTime();
result.setResponseTime(300);
```

this.responseTime 表示当前对象的属性；右侧 responseTime 表示传入方法的参数。

五、在 setter 中加入参数校验

```
public void setResponseTime(int responseTime) {
    if (responseTime < 0) {
        System.out.println("Invalid response time");
        return;
    }
    this.responseTime = responseTime;
}
```

当传入值小于 0 时，return 会直接结束方法，后面的赋值语句不会执行。因此非法值不会写入对象。

六、完整示例：HttpResult.java

```
public class HttpResult {
    private int responseTime;
    private int statusCode;

    public HttpResult(int responseTime, int statusCode) {
        setResponseTime(responseTime);
        setStatusCode(statusCode);
    }

    public int getResponseTime() {
        return responseTime;
    }

    public void setResponseTime(int responseTime) {
        if (responseTime < 0) {
            System.out.println("Invalid response time");
            return;
        }
        this.responseTime = responseTime;
    }

    public int getStatusCode() {
        return statusCode;
    }

    public void setStatusCode(int statusCode) {
        if (statusCode < 100 || statusCode > 599) {
            System.out.println("Invalid HTTP status code");
            return;
        }
        this.statusCode = statusCode;
    }

    public boolean isSuccess() {
        return statusCode >= 200 && statusCode < 300;
    }

    public boolean isSlow() {
        return responseTime > 500;
    }

    public void showResult() {
        System.out.println("Response time: " + responseTime + "ms");
        System.out.println("Status code: " + statusCode);
        System.out.println("Success: " + isSuccess());
        System.out.println("Slow request: " + isSlow());
    }
}
```

七、Main.java 调用示例

```
public class Main {
    public static void main(String[] args) {
        HttpResult result = new HttpResult(300, 200);

        System.out.println("Original time: " + result.getResponseTime());
        result.setResponseTime(800);
        result.setResponseTime(-100);

        result.showResult();
    }
}
```

预期关键结果：原始响应时间为 300；非法的 -100 会被拒绝；最终响应时间为 800ms；状态码为 200；请求成功判断为 true；慢请求判断为 true。

八、性能测试场景中的封装价值

性能测试结果对象可能承载大量请求的响应时间和状态码。若任何代码都能随意修改属性，非法值可能污染统计结果。把属性设为 private，并在 setter 中统一校验，可以在数据进入对象时建立基本质量控制。数据对象负责保证基本数据质量，统计方法负责基于数据进行计算。

本例状态码采用 100～599 的常见范围作为示范校验规则；具体项目可按协议和业务约定调整，不应把示例限制机械套用到所有系统。

九、课后作业

1. 独立编写 HttpResult 类，将 responseTime 和 statusCode 都改成 private。

2. 为两个属性分别编写 getter 和 setter。

3. responseTime 小于 0 时拒绝赋值；statusCode 不在 100～599 范围时拒绝赋值。

4. 保留 isSuccess()、isSlow() 和 showResult() 方法，并确保它们通过对象内部属性完成判断。

5. 在 Main 中创建对象，依次设置 300ms、800ms、-100ms，观察非法值是否被拦截。

6. 思考题：为什么不建议把属性全部声明为 public？只提供 getter、不提供 setter，适合什么场景？

十、验收标准

> 能够解释 private 与 public 的区别；能独立编写并调用 getter/setter；能说明 this 的含义；能用 setter 拦截非法响应时间；能够解释封装对性能测试数据质量的帮助。关键点未掌握前，先修正作业，不进入 Day 9。

提交方式：上传 HttpResult.java 和 Main.java，或粘贴完整代码与运行结果。批改时会检查代码，并在可用环境下尝试编译运行。